

Imposing a Linear Control Law Through MPC

Frank Vilimek

July 2026

1 Introduction

MPC, by design, does not use a linear control law, where the control input for the very next time step is calculated based on the current state. Instead, it calculates the optimal series of discrete control inputs from now until a finite end point in the near future, which may or may not be identical to the trajectory obtained through a linear controller. However, because MPC's control input is calculated such that it is optimal, it does not have an explicit form, making it hard to make the guarantees that explicit control laws allow. We believe it could be potentially useful to impose a linear control law-like constraint to MPC such that it roughly follows a trajectory similar to a linear controller, where we could expand upon the linear control law theory to make predictions on the MPC's closed-loop behavior, such as performance.

2 Linear Controller to Emulate: LQR

It must come as no surprise that the linear controller that we choose to emulate with MPC is LQR. Being perhaps the most similar explicit linear controller to MPC, LQR minimizes a similar cost function to MPC and thus produces optimal state and inputs trajectories, with the caveat being that there is no constraint handling. As a reminder, LQR is a form of optimal control that minimizes the cost function,

$$J = \int_0^\infty (x^\top Q x + u^\top R u) dt, \quad (1)$$

where x is the state vector, u is the input vector, Q is the state cost matrix, and R is the input cost matrix. LQR uses a linear control law that depends on the state,

$$u = -Kx, \quad (2)$$

where K is the controller gain. Linear control laws are not specific to LQR, and K can be calculated using other methods (pole placement/full-state feedback, for example); however, LQR is special in that it determines the single K that minimizes the cost function in equation (1)—any other method for calculating K does not guarantee this cost minimization.

Depending on your specific formulation of LQR (continuous vs. discrete, finite horizon vs. infinite horizon, crossover vs. no crossover, etc.), the specific K can also change, but the idea remains the same. In the most typical case, LQR refers to infinite horizon, continuous LQR, where the cost is calculated via an

integral from $t = 0$ to $t \rightarrow \infty$. In this case, K is calculated through the expression,

$$K = R^{-1}B^{\top}P, \quad (3)$$

where P is the solution to the continuous-time, algebraic Riccati equation (CARE),

$$A^{\top}P + PA - PBR^{-1}B^{\top}P + Q = 0. \quad (4)$$

An interesting fact about LQR is that it not only minimizes the cost function to obtain optimal state and input trajectories, but it also minimizes the $\mathcal{H}_2$ -norm, a common metric used to quantify the performance of a closed-loop system that has a form nearly identical to the LQR cost function. Thus, it is often found that LQR provides the most optimal and highest-performing trajectories.

3 MPC Review

MPC, as a quick review, is another form of optimal control that outshines the rest due to its strength in constraint handling paired with minimization of a cost function. By design and necessity, MPC is a discrete controller that operates step-wise, and in between steps it uses a zero-order hold (ZOH), meaning that the control input is kept constant in between the discrete inputs. The discrete cost function that MPC minimizes is extremely similar to the LQR cost function,

$$J = x_N^{\top}Px_N + \sum_{k=0}^{N-1} (x_k^{\top}Qx_k + u_k^{\top}Ru_k) \quad (5)$$

where P is now the terminal state cost, which is often used to help the optimization converge. However, typically for MPC, the control input isn't calculated via a linear relationship with the state and some controller gain; instead, it is calculated using quadratic programming, an optimization technique that finds the series of control inputs along a multi-step trajectory that obtains the minimum cost, while taking into consideration any state or input constraints. The optimization problem it solves is,

$$\begin{aligned} \min_{\mathcal{U}} \quad & \left\{ \frac{1}{2} \mathcal{U}^{\top} H \mathcal{U} + q^{\top} \mathcal{U} \right\} \\ \text{s.t.} \quad & G \mathcal{U} \leq W + T x_0 \end{aligned} \quad (6)$$

where practically all the quantities are matrices containing the dynamics of the system, state constraints, and/or input constraints. It is quite involved to just come up with the MPC problem to solve.

It is the constraint handling that makes MPC so powerful, yet it is also the very thing that makes it difficult to use. There is no explicit, closed-form equation for MPC (and explicit MPC is out of the question), so any result that we find using a linear controller such as LQR cannot easily be translated to MPC, if it is even possible. However, one of the benefits of MPC is that the states can be augmented to include really anything you want (which is a feature of model-based control methods in general), and the constraints can be made to essentially make the controller behave however you want. With the right combination of states and constraints, the closed-loop behavior can potentially be quite flexible.

4 MPC Formulation

4.1 Nonlinear Dynamic Model: Two-State Non-Isothermal CSTR

To much dismay, there is surprisingly not too much literature about employing a linear control law with MPC—the closest I could find were [1, 2, 3, 4], though they still don't exactly answer what I'm looking for. We will just have to come up with something ourselves and see if it works.

We start off by considering the continuous-time, non-isothermal CSTR model containing the reaction $A \rightarrow B$, where the manipulated variable is the inlet volumetric flow rate F :

$$\begin{aligned}\frac{\partial C_A}{\partial t} &= \frac{F}{V} (C_{A0} - C_A) - A_0 e^{-E/RT} C_A \\ \frac{\partial T}{\partial t} &= \frac{F}{V} (T_0 - T) + \frac{(-\Delta H)}{\rho C_p} A_0 e^{-E/RT} C_A + \frac{UA}{\rho C_p V} (T_c - T)\end{aligned}\quad (7)$$

We consider that, since F is the manipulated variable, we can equivalently say that,

$$u = F = -Kx \quad (8)$$

where $x = [C_A \ T]^\top$ and $K = [k_1 \ k_2]$. Substituting $F = -Kx$ then multiplying out the vectors, we are left with the dynamic model,

$$\begin{aligned}\frac{\partial C_A}{\partial t} &= -\left(\frac{k_1 C_A + k_2 T}{V}\right) (C_{A0} - C_A) - A_0 e^{-E/RT} C_A \\ \frac{\partial T}{\partial t} &= -\left(\frac{k_1 C_A + k_2 T}{V}\right) (T_0 - T) + \frac{(-\Delta H)}{\rho C_p} A_0 e^{-E/RT} C_A + \frac{UA}{\rho C_p V} (T_c - T)\end{aligned}\quad (9)$$

4.2 LTI State-Space Model

To make things easier for us right now, let us linearize it around a stable control input (which, again, is now the gain matrix K). A natural choice for this is the LQR gain, which requires us to consider the model where we did not make the $F = -Kx$ substitution. As usual, the A and B matrices are,

$$A = \begin{bmatrix} -\left(\frac{F_{A0}^{eq}}{V} + A_0 e^{-E/RT_{eq}}\right) & \frac{A_0 E}{RT_{eq}^2} e^{-E/RT_{eq}} C_A^{eq} \\ \frac{(-\Delta H)}{\rho C_p} A_0 e^{-E/RT_{eq}} & \left(-\frac{(-\Delta H) A_0 E}{\rho C_p RT_{eq}^2} e^{-E/RT_{eq}} C_A^{eq} - \frac{F_{A0}^{eq}}{V} - \frac{UA}{\rho C_p V}\right) \end{bmatrix}, \quad B = \begin{bmatrix} \frac{C_{A0} - C_A^{eq}}{V} \\ \frac{T_0 - T_{eq}}{V} \end{bmatrix}. \quad (10)$$

Obtaining the LQR gain from this, which we will call K_{LQR} , provides us with a sort of stable operating K that we know has favorable performance qualities. We can use this to build the actual LTI state-space model for the system where K is the control input, where we just have to find the Jacobian of equation (9) with respect to the states $x = [C_A \ T]^\top$ and inputs $u = K^\top = [k_1 \ k_2]^\top$ (where we now make the $F = -Kx$ substitution as we did in equation (9)). This will yield us the A_c and B_c matrices,

$$A_c = \left. \frac{\partial f}{\partial x} \right|_{x_{eq}, u_{eq}, K_{LQR}}, \quad B_c = \left. \frac{\partial f}{\partial u} \right|_{x_{eq}, u_{eq}, K_{LQR}}, \quad (11)$$

which, after discretization, will allow us to construct the discrete-time state-space model,

$$x_{k+1} = A_d x_k + B_d u_k. \quad (12)$$

4.3 Augmented State Vector

We now can construct an augmented state vector that includes the errors between the states x as well as the gain relative to K_{LQR} :

$$z = \begin{bmatrix} C_A - C_A^{eq} \\ T - T_{eq} \\ k_1 - k_{1,LQR} \\ k_2 - k_{2,LQR} \end{bmatrix} = \begin{bmatrix} e_{C_A} \\ e_T \\ e_{k_1} \\ e_{k_2} \end{bmatrix} \quad (13)$$

which produces the state-space model,

$$\underbrace{\begin{bmatrix} e_{C_A,k+1} \\ e_{T,k+1} \\ e_{k_1,k+1} \\ e_{k_2,k+1} \end{bmatrix}}_{z_{k+1}} = \underbrace{\begin{bmatrix} a_{11} & a_{12} & 0 & 0 \\ a_{21} & a_{22} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{A_{aug}} \underbrace{\begin{bmatrix} e_{C_A,k} \\ e_{T,k} \\ e_{k_1,k} \\ e_{k_2,k} \end{bmatrix}}_{z_k} + \underbrace{\begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \\ 1 & 0 \\ 0 & 1 \end{bmatrix}}_{B_{aug}} \underbrace{\begin{bmatrix} k_1 \\ k_2 \end{bmatrix}}_{u_k}. \quad (14)$$

This state-space model now allows us to place a cost on the deviation of $K_{MPC} = [k_1 \ k_2]^\top$ from K_{LQR} , meaning we can allow K_{MPC} to take values different from K_{LQR} (for example, when the states reach a point near their constraints), but when constraints don't need to be handled, $K_{MPC} \rightarrow K_{LQR}$.

5 Results and Discussion

The CSTR system was modeled with a sampling period of $T_s = 10$ and an MPC prediction horizon $N = 10$. The state cost matrix Q was chosen as,

$$Q = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.1 & 0 & 0 \\ 0 & 0 & 100 & 0 \\ 0 & 0 & 0 & 1000 \end{bmatrix}$$

to more evenly penalize the states as they all vary greatly in orders of magnitude. Input constraints were also chosen as $u_k \in [-0.005, 0.005]$, simply so that the resulting profiles look cool. Below are the concentration, temperature, and gain profiles after experiencing an impulse disturbance in all the states to probe the system's behavior.

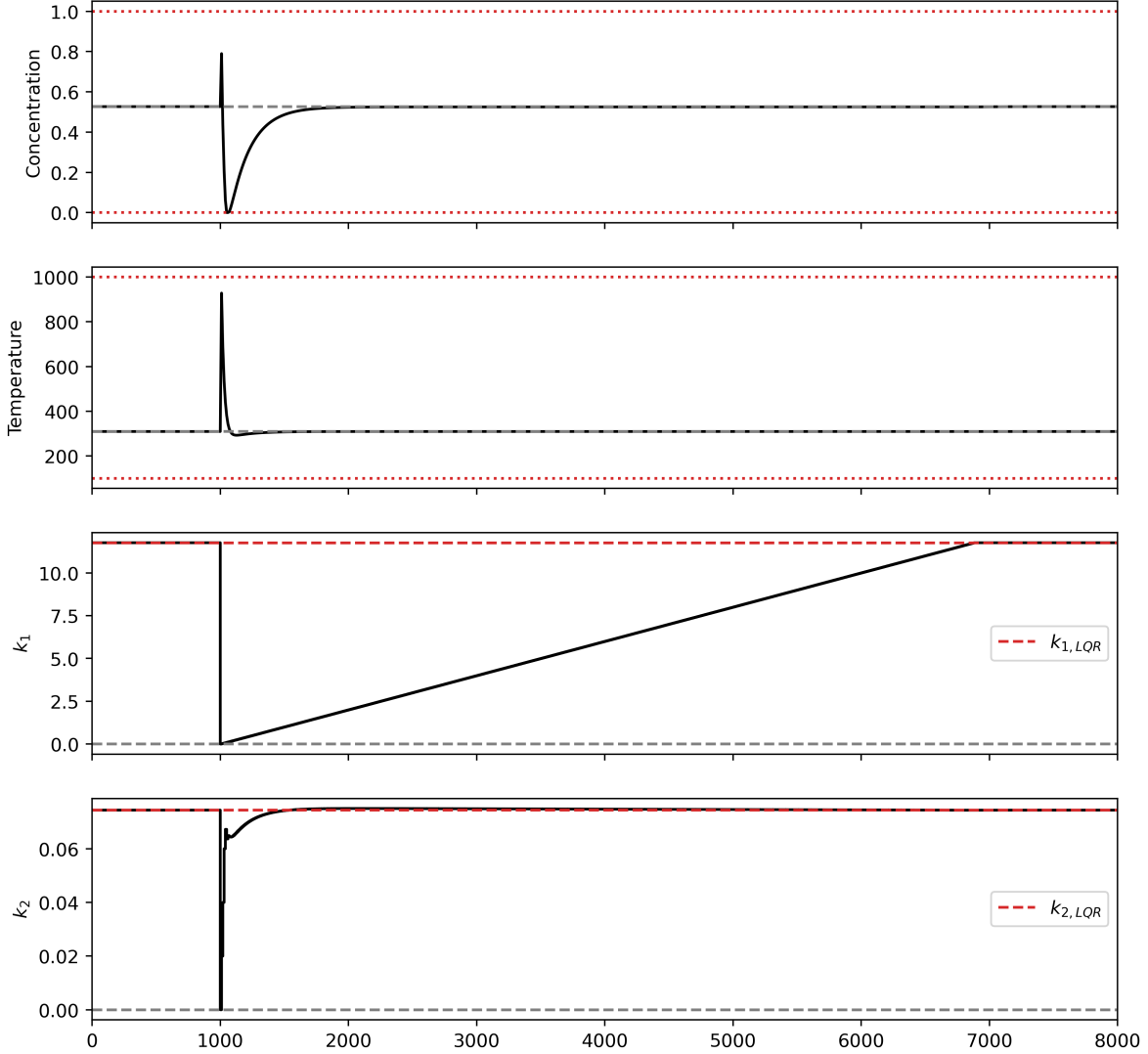


Figure 1: Concentration, temperature, and gain profiles over time.

We can see that through changing the gain, MPC is more able to modulate the concentration and temperature profiles to steer away from their respective constraints, then when the states reach their equilibrium points, K_{MPC} slowly changes to K_{LQR} . With the chosen Q in figure (1) the time-varying nature of K_{MPC} is apparent, but unrealistic because it moves so slowly. Figure (2) shows a more realistic example, where $K_{MPC} = K_{LQR}$ for all times when the states are not near their constraints, but when the concentration nears the lower constraint, there is a small blip seen in k_2 , indicating that the controller must adjust the gain to prevent violation of constraints. For this case, Q was chosen such that the deviation $K_{MPC} - K_{LQR}$ had a very large penalty, forcing the controller to have a gain as close to K_{LQR} as possible, and only changing it when it needs to.

Plant-model mismatches were also experimented with to see how robust the control system is. Though this is not an entirely useful investigation, it's quite interesting to see the limit to which this control method can be applied. Figure (3) shows some example state profiles when a small plant-model mismatch is present, showing that though it is more unstable than the model with no mismatch, it is still able to be controlled.

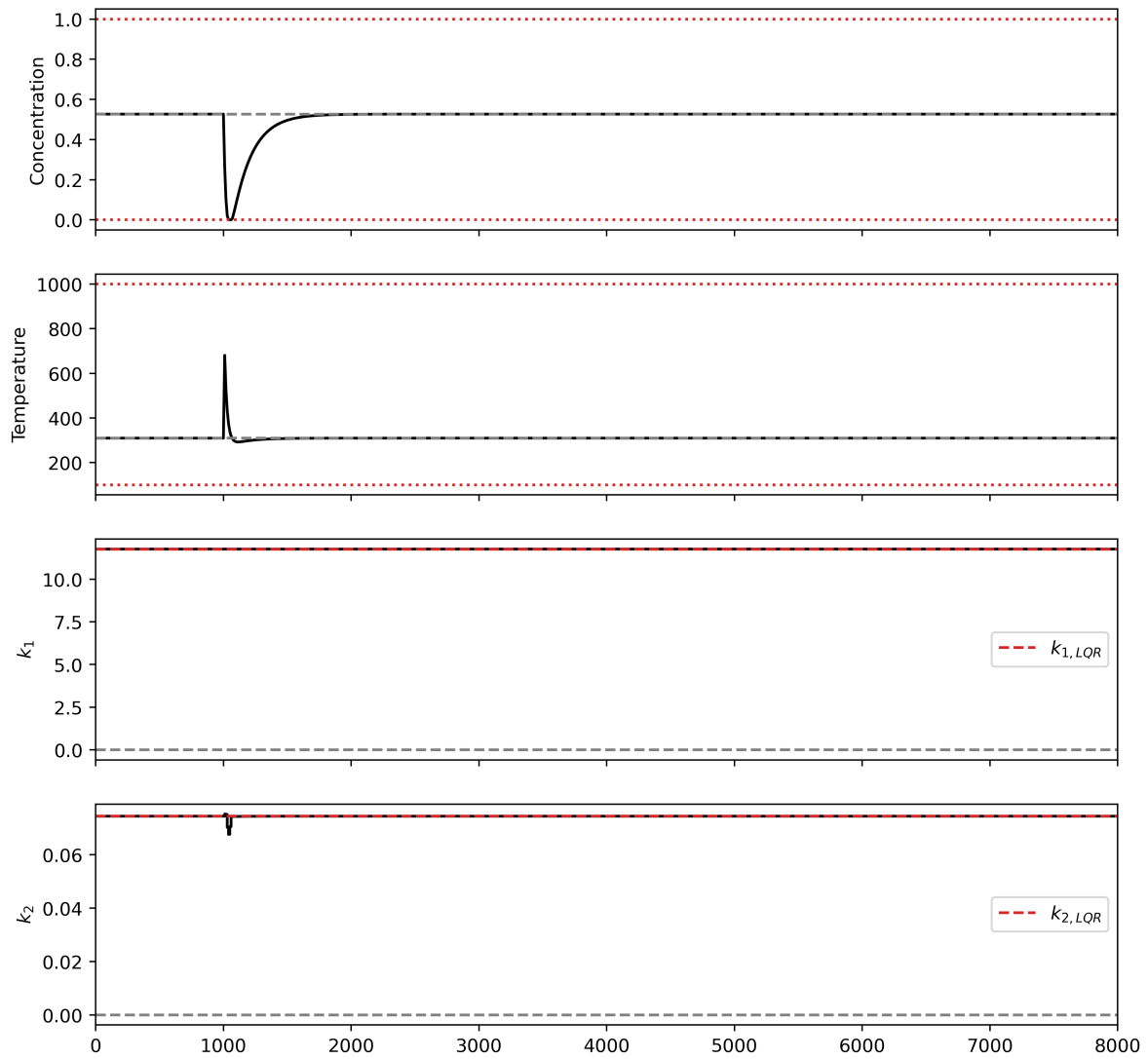


Figure 2: Concentration, temperature, and gain profiles over time with large penalties placed on K_{MPC} .

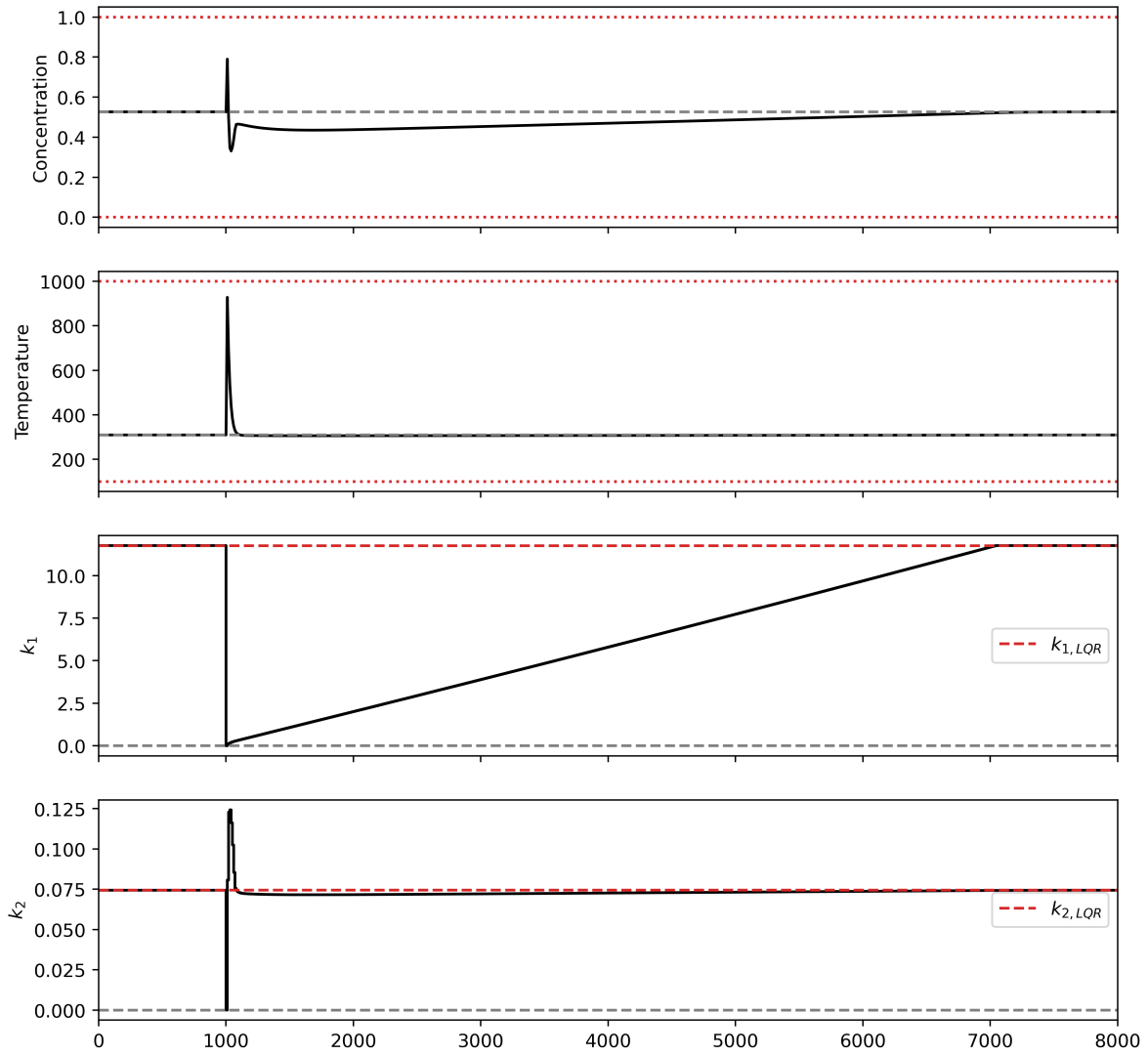


Figure 3: Concentration, temperature, and gain profiles over time with a plant-model mismatch.

6 Conclusion

These results of this study are promising, showing that imposing a linear control law through MPC is possible. This allows us to start thinking about bridging the gap between explicit LQR/pole placement and implicit MPC. Explicit control laws allow us to make guarantees on stability and performance, and if MPC is constrained to operate according to those laws, we may be able to characterize MPC's stability and performance based on the results we get from the explicit systems.

References

- [1] M. S. Darup, M. Jost, G. Pannocchia, and M. Mönnigmann, "On the maximal controller gain in linear mpc," *IFAC-PapersOnLine*, vol. 50, p. 9218–9223, Jul 2017.
- [2] B. Kouvaritakis, M. Cannon, and J. Rossiter, "Who needs qp for linear mpc anyway?," *Automatica*, vol. 38, p. 879–884, May 2002.
- [3] D. Limon, J. M. Gomes da Silva, T. Alamo, and E. Camacho, "Improved mpc design based on saturating control laws," *2003 European Control Conference (ECC)*, p. 707–712, Sep 2003.
- [4] M. Mönnigmann, J. Otten, and M. Jost, "Nonlinear mpc defines implicit regional optimal control laws," *IFAC-PapersOnLine*, vol. 48, no. 23, p. 142–147, 2015.